

Autochess: Sistema Físico Interativo de Xadrez com Machine Learning Educacional

Fernando Frare Vieira*
Gabriel Affonso Borges Caballero**
Marco Vieira Busetti***

**Engenharia de Computação, Universidade Tecnológica Federal do Paraná, (e-mail: fernandofrarevieira@alunos.utfpr.edu.br)*

***Engenharia de Computação, Universidade Tecnológica Federal do Paraná, (e-mail: gabrielaffonso@alunos.utfpr.edu.br)*

****Engenharia Elétrica, Universidade Tecnológica Federal do Paraná, (e-mail: marcovieirabusetti@alunos.utfpr.edu.br)*

Abstract: This paper presents a physical and interactive chess system designed to teach artificial intelligence concepts to children. The project uses a reduced 4×4 chessboard where users play against a Q-Learning-based AI that adapts its strategy throughout the game. Piece movement is handled by a CoreXY manipulator equipped with Nema 17 stepper motors and an SG90 servo motor, controlled by an Arduino UNO running GRBL-Servo firmware. Board state recognition is performed through computer vision using the SIFT (Scale-Invariant Feature Transform) algorithm.

Resumo: Este trabalho apresenta um sistema físico e interativo de xadrez, desenvolvido com o objetivo de ensinar conceitos de inteligência artificial para crianças. O projeto utiliza um tabuleiro reduzido 4x4, no qual usuários jogam contra uma IA baseada em Q-Learning que se adapta durante o jogo. A movimentação física utiliza um manipulador CoreXY com motores Nema 17 e um servo SG90, controlados por Arduino UNO com firmware GRBL-servo. A detecção da configuração das peças no tabuleiro é feita por meio de visão computacional, utilizando o algoritmo SIFT.

Keywords: Machine Learning; Computer Vision; Automation; Chess; Technology Education; Q-Learning; SIFT; CoreXY.

Palavras-chaves: Machine Learning; Visão Computacional; Automação; Xadrez; Educação Tecnológica; Q-Learning; SIFT; CoreXY.

1. INTRODUÇÃO

Na última década, a popularização de tecnologias como a inteligência artificial e a automação têm gerado avanços e transformações em diversas áreas do conhecimento, incluindo a educação [1]. Em paralelo, cresce a necessidade de que as novas gerações compreendam esses conceitos, pois essas tecnologias estão cada vez mais presentes no cotidiano. Porém, apesar dessa crescente demanda, ainda existem diversos desafios na introdução desses temas complexos de forma acessível para públicos mais jovens, especialmente crianças em idade escolar.

Uma das dificuldades mais comuns é encontrar formas de ensinar os fundamentos da inteligência artificial de maneira prática e compreensível. Há uma carência de métodos educacionais que demonstrem o funcionamento de uma IA de forma interativa e visível, permitindo que as crianças observem concretamente como ocorre o processo de aprendizado e tomada de decisão computacional.

O xadrez, historicamente utilizado como benchmark para avaliar sistemas de inteligência artificial [2], oferece um contexto ideal para demonstração desses conceitos. Desde

o desenvolvimento do Deep Blue da IBM até sistemas mais recentes como o AlphaZero, o xadrez tem servido como plataforma para avanços significativos em IA. Contudo, a complexidade do xadrez tradicional (com aproximadamente 10^{120} possíveis jogos) torna impraticável sua implementação em sistemas educacionais com recursos limitados.

Diante desse cenário, este trabalho propõe uma solução lúdica e educativa utilizando uma variante simplificada do xadrez (Minichess 4×4) como meio para introduzir conceitos de inteligência artificial, visão computacional e automação. O projeto também visa estimular o raciocínio lógico e a curiosidade tecnológica através da interação direta com um sistema físico autônomo [3].

Este artigo apresenta o desenvolvimento do projeto Autochess, que consiste em um tabuleiro de xadrez físico 4×4 que integra módulos de visão computacional baseada em SIFT, inteligência artificial utilizando Q-Learning e um sistema CNC controlado por Arduino com firmware GRBL-servo. O objetivo é demonstrar como o aprendizado de máquina pode ser implementado de forma prática no processo educativo, incentivando o interesse por tecnologia desde os estágios iniciais da formação escolar.

2. TRABALHOS RELACIONADOS

A intersecção entre jogos, inteligência artificial e educação tem sido explorada por diversos pesquisadores. Silver et al. [4] demonstraram o potencial do aprendizado por reforço no contexto de jogos através do AlphaGo, que utilizou técnicas de aprendizado profundo combinadas com busca em árvore Monte Carlo. Embora aplicado ao jogo de Go, os princípios fundamentais são relevantes para sistemas educacionais baseados em jogos.

No contexto específico de variantes simplificadas de xadrez, Gardner [5] formalizou diversas modalidades de Minichess, incluindo a variante Silverman 4×4 utilizada neste trabalho. Essa simplificação permite análise completa do espaço de estados e facilita a implementação de algoritmos de aprendizado, tornando-se ideal para fins educacionais.

Sistemas físicos para ensino de robótica e automação têm ganhado destaque na literatura. Benitti [6] realizou uma revisão sistemática sobre o uso de robótica educacional, destacando os benefícios de sistemas físicos interativos no desenvolvimento de habilidades STEM (*Science, Technology, Engineering and Mathematics*). O presente trabalho contribui para essa área ao integrar conceitos de IA, visão computacional e automação em um único sistema educacional.

Especificamente sobre sistemas de xadrez automatizados, diversos projetos têm sido desenvolvidos utilizando diferentes abordagens tecnológicas. Contudo, a maioria foca em aspectos puramente técnicos, sem considerar o potencial educacional ou a demonstração visual do processo de aprendizado da IA, lacuna que este trabalho pretende preencher.

3. MATERIAIS E ORÇAMENTO

O desenvolvimento do sistema Autochess priorizou a utilização de componentes acessíveis e amplamente disponíveis no mercado nacional e internacional, visando demonstrar a viabilidade econômica do projeto para aplicações educacionais. A Tabela 1 apresenta a relação completa dos materiais utilizados e seus respectivos custos.

Item	Qtd.	Preço (R\$)
Arduino Uno	2	40,00
Shield CNC V3 + Drivers	1	15,00
Eletroimã	1	6,00
Motor Nema 17	2	40,00
Servo motor SG90	1	5,00
Placa universal	1	12,00
Patolas	2	30,00
Fonte de alimentação	2	50,00
Webcam Full HD	1	50,00
Impressão 3D	-	100,00
Base MDF	-	40,00
Outros (fios, conectores, etc)	-	30,00
Total		418,00

Tabela 1. Relação de materiais e custos do projeto Autochess.

O projeto foi desenvolvido com um orçamento acessível, utilizando componentes facilmente encontrados no mercado nacional e internacional, o que resulta em um custo-benefício adequado para fins educacionais.

4. METODOLOGIA

Esta seção descreve os procedimentos adotados para o desenvolvimento do projeto Autochess. Serão apresentados a visão geral do sistema, os aspectos mecânicos envolvidos na construção da estrutura, os componentes de hardware utilizados, e o software desenvolvido para os módulos de IA e visão computacional.

4.1 Visão geral

O Autochess é um sistema interativo que integra diferentes módulos tecnológicos para permitir a realização de partidas de xadrez em um ambiente físico com controle automatizado. A arquitetura do projeto foi planejada para que a interação entre o usuário e o sistema ocorra de forma contínua, envolvendo visão computacional, inteligência artificial e execução dos movimentos através de um sistema CNC.

O ciclo de funcionamento do sistema inicia-se com a realização de um movimento pelo jogador humano no tabuleiro físico. Após concluir sua jogada, o usuário pressiona um botão indicando que o sistema pode proceder à análise. Então uma câmera realiza a captura de uma imagem do tabuleiro, a qual é processada por algoritmos de Visão Computacional para identificar a nova configuração das peças. A posição atual do jogo é convertida em uma representação matricial, que é enviada como entrada para o módulo de Inteligência Artificial. A IA processa esta informação e determina o próximo movimento da máquina, retornando as coordenadas da jogada a ser realizada. Por fim, o comando correspondente é transmitido via comunicação serial para o sistema CNC, que executa automaticamente a movimentação física da peça no tabuleiro, completando o ciclo de interação. A Figura 1 ilustra o fluxo geral de funcionamento do sistema.

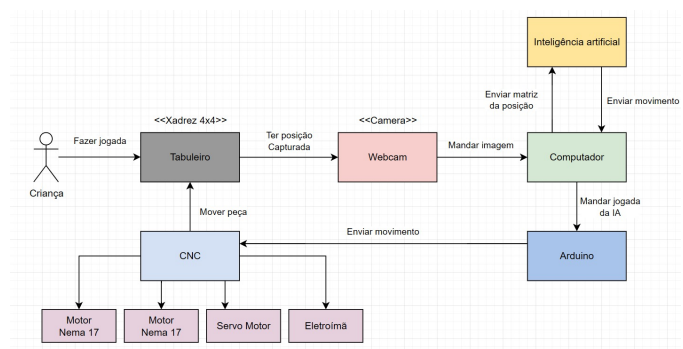


Figura 1. Diagrama geral do funcionamento do projeto.

Fonte: Autoria própria.

4.2 Hardware Mecânico

A parte mecânica do projeto consiste em um sistema de movimentação das peças de xadrez baseado em arquitetura CoreXY, além de estruturas de suporte para os demais elementos do sistema, como a câmera utilizada para visão computacional. A escolha da arquitetura CoreXY foi motivada por suas vantagens em termos de eficiência no uso do espaço, precisão de movimentação e velocidade de operação.

Durante a fase de desenvolvimento, toda a estrutura foi previamente modelada em software de modelagem tridimensional, permitindo a visualização detalhada da montagem antes da construção física. O modelo 3D completo está apresentado na Figura 2.

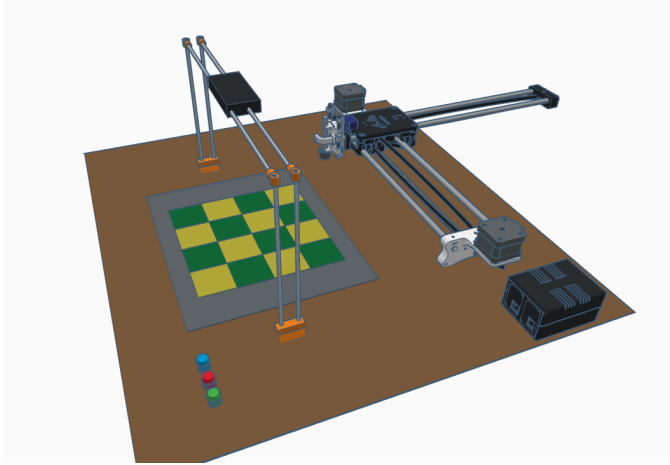


Figura 2. Modelo 3D do sistema completo.
Fonte: Autoria própria.

A estrutura principal do CoreXY foi baseada no projeto open-source DrawingBot da MakerC [8], disponibilizado através da plataforma Thingiverse. As peças estruturais foram fabricadas através de impressão 3D em material PLA, oferecendo leveza e facilidade de produção. As peças de xadrez impressas em 3D também se mostraram adequadas para comportar uma moeda de ferro magnetizada, utilizado para a atração via eletroímã das peças. Os arquivos originais foram adaptados para comportar os componentes específicos deste projeto, incluindo o suporte da câmera e o mecanismo de fixação do eletroímã.

Todos os componentes foram montados sobre uma base de MDF cortada a laser, oferecendo uma superfície de apoio para a fixação dos motores e suportes. O uso do MDF contribui tanto para a redução de custos quanto para a leveza do conjunto, facilitando o transporte e manuseio do sistema. A Figura 3 apresenta o design da base de MDF utilizada no projeto.

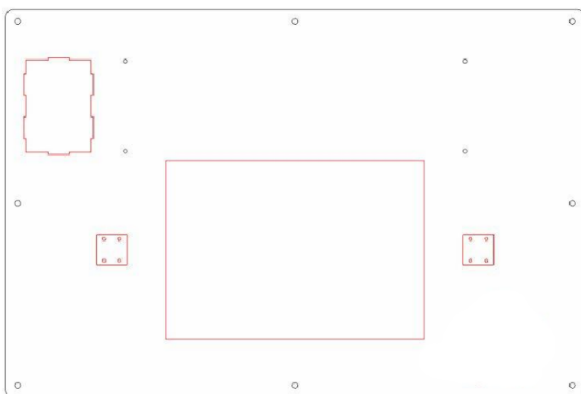


Figura 3. Design da base de MDF cortada a laser.
Fonte: Autoria própria.

A movimentação nos eixos X e Y é realizada por dois motores de passo Nema 17, com torque de 4,2 kg·cm e precisão de 1,8° por passo. A transmissão do movimento é feita através de uma correia dentada GT2 e quatro polias sincronizadas de 20 dentes, dispostas segundo o padrão cruzado característico do sistema CoreXY. Esta configuração permite que ambos os motores contribuam para o movimento em qualquer direção, resultando em maior velocidade e precisão.

O movimento do eixo Z foi implementado utilizando um servo motor SG90, que aciona um mecanismo de elevação e descida do eletroímã. O eletroímã utilizado possui força de atração de aproximadamente 2,5 kg a 12V, suficiente para manipular as peças de xadrez. A estrutura CoreXY implementada está ilustrada na Figura 4.

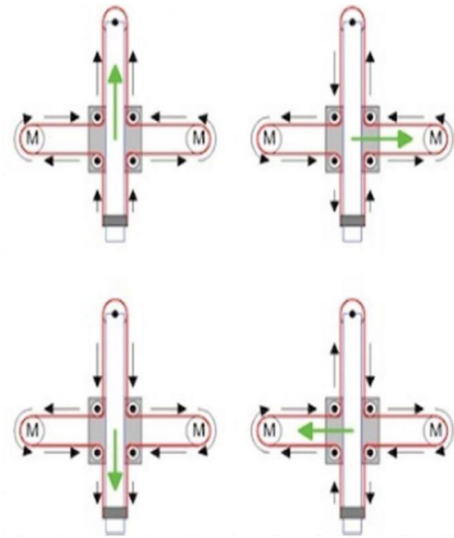


Figura 4. Estrutura cruzada CoreXY utilizada no projeto.
Fonte: Tested 3D Prints [7].

Para a captura de imagens, foi desenvolvido um suporte específico para câmera, modelado e impresso em 3D, que posiciona a webcam Full HD perpendicularmente sobre o tabuleiro a uma altura adequada para visualização completa da área de jogo, conforme mostrado na Figura 5.

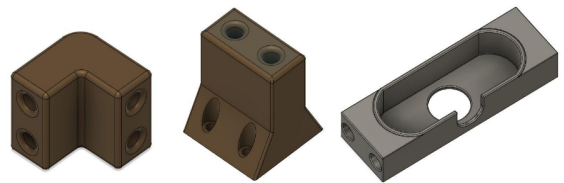


Figura 5. Suporte para câmera desenvolvido para o projeto.
Fonte: Autoria própria.

As peças de xadrez também foram especialmente projetadas para o projeto, incluindo moedas de ferro magnetizadas em suas bases para permitir a manipulação magnética. A Figura 6 apresenta o modelo da rainha e sua base, a qual possui uma abertura para inserção de uma moeda de ferro magnetizada.



Figura 6. Modelo 3D da peça rainha com sua base.
Fonte: Autoria própria.

4.3 Hardware Eletrônico

A implementação do hardware eletrônico envolveu a integração de diversos componentes, responsáveis pela comunicação entre os módulos de software e o controle do sistema CNC. Foi utilizado um computador que atua como o cérebro do sistema, executando a lógica principal da inteligência artificial e coordenando os módulos periféricos. Ele se comunica com dois Arduinos por meio de conexões seriais via USB, delegando o controle físico da movimentação CNC e da interface com o usuário.

O projeto elétrico do sistema de movimentação da CNC está apresentada na Figura 7.

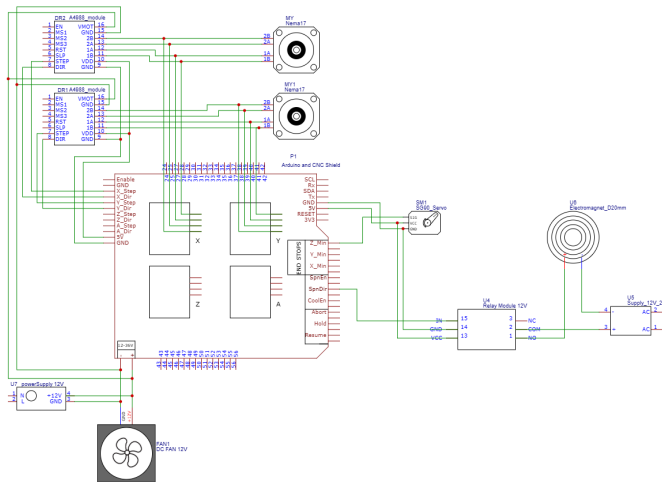


Figura 7. Diagrama do projeto elétrico do sistema.
Fonte: Autoria própria.

Para o controle da movimentação CNC, foi utilizado um Arduino UNO executando o firmware GRBL-servo, uma versão modificada do projeto open-source GRBL disponibilizado por Robottini [10] no Github. O GRBL-servo é capaz de interpretar comandos G-code padrão para movimentação dos motores de passo, com a extensão adicional de controle de servo motor. Durante a implementação, o firmware foi modificado para ajustar os ângulos específicos de abertura e fechamento do servo motor, permitindo o controle preciso das operações de captura e soltura das peças.

O driver utilizado para os motores de passo é o shield CNC V3, que incorpora drivers A4988 para cada motor. Este shield oferece microstepping configurável (até 1/16 de passo), permitindo movimentação suave e precisa. A alimentação dos motores é fornecida por uma fonte externa de 12V/5A, isolada do circuito de controle do Arduino.

O segundo Arduino UNO foi integrado ao sistema para gerenciar a interface com o usuário através de três botões

físicos. O primeiro botão deve ser pressionado pelo jogador após concluir sua jogada, sinalizando ao sistema para iniciar a captura e processamento da imagem do tabuleiro. O segundo botão permite ao usuário indicar o término da partida, ativando a rotina de finalização. O terceiro botão tem a função de reinicializar a inteligência artificial, apagando todo o conhecimento acumulado (tabela Q) e retornando a IA ao estado inicial de aprendizado.

A decisão de utilizar um segundo Arduino dedicado aos botões foi tomada para evitar sobrecarga no Arduino da CNC, que já opera com a CNC Shield V3 e o firmware GRBL-servo, exigente em termos de temporização e controle; integrar os botões em um LED RGB ao mesmo microcontrolador poderia causar conflitos de hardware e dificultar a manutenção do sistema.

4.4 Software

O desenvolvimento do software para o projeto consiste na implementação de dois módulos principais: a Inteligência Artificial responsável por decidir os movimentos da máquina e a Visão Computacional encarregada da captura e interpretação da posição das peças no tabuleiro. Ambos os módulos foram desenvolvidos em Python, utilizando bibliotecas especializadas para cada função. A integração entre a visão computacional, a inteligência artificial e o sistema de movimentação foi realizada por meio de um programa desenvolvido em Python, denominado *Core*.

4.4.1 Inteligência Artificial: Para reduzir a complexidade computacional e viabilizar o treinamento da IA em tempo hábil, optou-se por utilizar uma variante simplificada do xadrez tradicional, conhecida como Silverman 4×4. Essa modalidade, pertencente à família de jogos Minichess, é jogada em um tabuleiro 4×4 e conta com um conjunto reduzido de peças: rei, rainha, torres e peões para cada jogador, conforme mostrado na Figura 8.

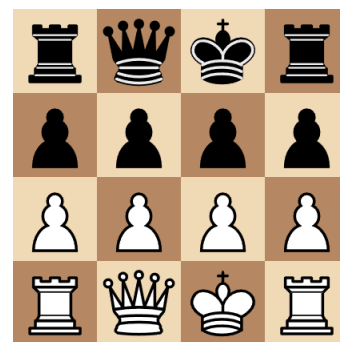


Figura 8. Configuração inicial do tabuleiro Silverman 4×4.
Fonte: Wikipedia [9]

A configuração inicial do tabuleiro Silverman 4×4 posiciona as peças brancas na primeira fileira (rainha em a1, rei em b1, torre em c1 e peão em d1) e na segunda fileira (peões em a2, b2, c2, d2). As peças pretas ocupam posições simétricas nas fileiras 3 e 4. Esta configuração reduzida mantém a essência estratégica do xadrez enquanto permite análise computacional viável.

A IA foi desenvolvida utilizando a técnica de Aprendizado por Reforço, através do algoritmo Q-Learning, modelado como um Processo de Decisão de Markov (MDP) [11].

- **Estados (S):** Cada estado representa uma configuração única do tabuleiro, codificada como uma matriz 4×4 onde cada célula contém a identificação da peça presente;
- **Ações (A):** Conjunto de todos os movimentos legais possíveis em um dado estado, respeitando as regras de movimento de cada tipo de peça;
- **Função de Recompensa (R):** Definida de forma simples como +1 para vitórias, 0 para empates e -1 para derrotas;
- **Política de Ação:** Implementada através de estratégia ϵ -gulosa para equilibrar exploração e exploração.

A equação (1) representa a atualização do Q-Learning [12]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (1)$$

onde $\alpha = 0.5$ é a taxa de aprendizado, $\gamma = 0.9$ é o fator de desconto, r é a recompensa recebida, s' é o novo estado e a' representa as ações possíveis no novo estado.

Os valores de α e γ foram escolhidos empiricamente após testes preliminares, visando garantir uma taxa de aprendizado moderada e boa retenção de conhecimento a longo prazo.

A estratégia ϵ -gulosa [12] foi implementada com decaimento linear, iniciando com $\epsilon = 0.9$ (alta exploração) e decrescendo gradualmente até $\epsilon = 0.01$ (predominante exploração), permitindo que a IA explore amplamente o espaço de estados nas fases iniciais de treinamento e posteriormente se concentre em explorar o conhecimento adquirido.

Para fins de validação e análise visual, foi desenvolvida uma interface gráfica utilizando a biblioteca PyGame, permitindo a simulação das partidas entre humanos e a IA, bem como partidas entre diferentes versões da IA para avaliação de desempenho. A interface implementada está ilustrada na Figura 9.

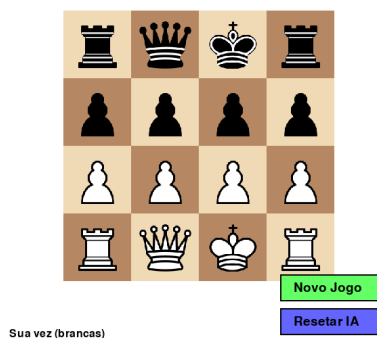


Figura 9. Interface gráfica do tabuleiro Minichess 4×4 desenvolvida em PyGame.

Fonte: Autoria própria

4.4.2 Visão Computacional: O módulo de visão computacional tem como função principal identificar automa-

ticamente o estado atual do tabuleiro após cada jogada do usuário. Este processo é iniciado com a captura de uma imagem através de uma webcam Full HD posicionada fixamente acima do tabuleiro, oferecendo uma visão perpendicular completa da área de jogo.

Durante a etapa de processamento, o sistema enfrenta diversos desafios técnicos, incluindo variações na iluminação ambiente, distorções ópticas da lente da câmera, reflexos na superfície das peças e a necessidade de distinguir com alta precisão as peças brancas e pretas sob diferentes condições de luminosidade.

A solução adotada foi a implementação do algoritmo **SIFT (Scale-Invariant Feature Transform)**, uma técnica robusta para detecção de pontos-chave e descrição de características locais invariantes à escala, rotação e iluminação [13]. O SIFT foi escolhido por sua comprovada eficácia em reconhecimento de objetos em condições adversas e sua capacidade de manter consistência mesmo com variações significativas nas condições de captura.

O processo de reconhecimento segue as seguintes etapas:

- (1) **Pré-processamento:** A imagem capturada é convertida para escala de cinza e submetida a filtros de suavização para redução de ruído;
- (2) **Segmentação do tabuleiro:** Utilização de detecção de contornos e transformação de perspectiva para isolar a região de interesse (tabuleiro) e corrigir distorções angulares;
- (3) **Divisão em células:** O tabuleiro segmentado é dividido matematicamente em 16 células (4×4), com cada célula sendo analisada individualmente;
- (4) **Extração de características SIFT:** Para cada célula, são extraídos pontos-chave e descritores SIFT que caracterizam as texturas e formas presentes;
- (5) **Matching com banco de referências:** Os descritores extraídos são comparados com um banco de imagens de referência previamente construído, contendo exemplares de cada tipo de peça em diferentes posições e condições de iluminação;
- (6) **Classificação e mapeamento:** Com base na similaridade dos descritores, cada célula é classificada como vazia ou contendo uma peça específica, gerando a matriz 4×4 que representa o estado atual do tabuleiro.

O banco de referências foi construído através da captura de múltiplas imagens de cada tipo de peça sob diferentes condições de iluminação e posicionamento, totalizando aproximadamente 50 imagens de referência por tipo de peça. Este processo garante robustez no reconhecimento mesmo com variações nas condições de captura durante o uso real do sistema.

As Figuras 10, 11 e 12 demonstram exemplos práticos da aplicação do algoritmo SIFT para reconhecimento das peças em diferentes condições, evidenciando a robustez da solução adotada.

Note que do lado direito da imagem está o resultado do tratamento realizado pelo SIFT. Temos as peças brancas denotadas como círculos brancos e peças pretas denotadas como círculos pretos. A inicial de cada peça se encontra no centro de cada círculo, onde P denota *Pawn* (Peão),

R denota *Rook* (Torre), *Q* denota *Queen* (Rainha) e *K* denota *King* (Rei).

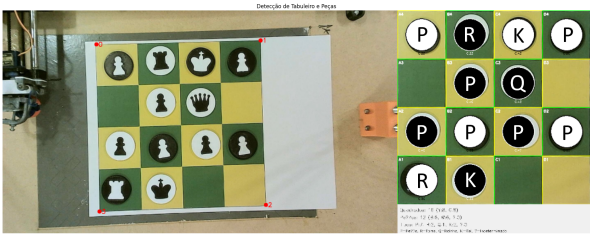


Figura 10. Exemplo 1 da aplicação do algoritmo SIFT para reconhecimento de peças.
Fonte: Autoria própria

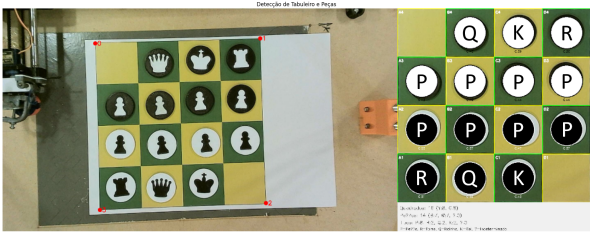


Figura 11. Exemplo 2 da aplicação do algoritmo SIFT para reconhecimento de peças.
Fonte: Autoria própria

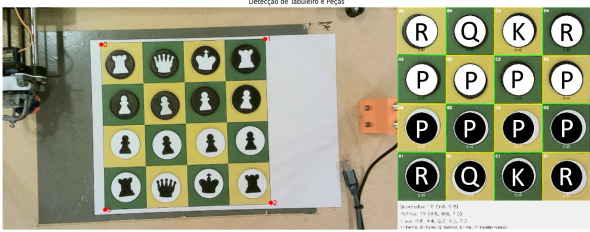


Figura 12. Exemplo 3 da aplicação do algoritmo SIFT para reconhecimento de peças.
Fonte: Autoria própria

4.4.3 Core: O *Core* do sistema consiste no script principal em Python, que roda em um computador, atuando como o orquestrador central, ao integrar os outros módulos do projeto. A lógica da partida é gerenciada pela classe *Minichess*, que mantém o estado atual do tabuleiro com uma matriz 4x4. Essa matriz interna serve como a "verdade" do sistema antes de cada jogada humana. O ciclo do script espera por uma entrada "0", "1" ou "2". Essas entradas servem para, respectivamente, confirmar uma jogada, começar um novo jogo e reiniciar a IA.

Ao receber o sinal, o *Core* aciona o módulo de CV para capturar e processar a imagem, que retorna uma nova matriz do estado do tabuleiro. Nesta etapa, um

tratamento de erros é executado: o script verifica se a nova configuração é resultado de um movimento válido a partir da matriz interna. Se não for válido, uma nova foto é tirada e processada. Isso ocorre até que haja um movimento válido, garantindo a integridade do jogo. Com a jogada humana validada, o *Core* a infere comparando a matriz antiga com a nova para obter as coordenadas de origem e destino.

Por fim, essa jogada é submetida à IA para determinar a resposta da máquina, e o movimento resultante é traduzido em comandos G-code e enviado ao controlador CNC para a execução física, completando o ciclo.

5. RESULTADOS E DISCUSSÃO

5.1 Avaliação da Inteligência Artificial

Para avaliar a eficácia do algoritmo Q-Learning implementado, foi desenvolvida uma metodologia sistemática de testes baseada em múltiplas sessões de treinamento. O protocolo experimental consistiu em 30 sessões distintas, cada uma com 20 partidas sequenciais, totalizando 600 partidas analisadas. A cada nova sessão, a inteligência artificial era reinicializada, garantindo a independência estatística dos dados coletados.

As métricas de avaliação adotadas foram:

- **Média de jogadas por partida:** Indicador da duração das partidas, refletindo a capacidade da IA de dificultar vitórias imediatas de partidas;
- **Número de *blunders* (erros graves):** Quantificação de erros críticos que resultam em perda imediata de material ou posição;
- **Saldo de material:** Relação de peças ganhas e peças perdidas pela IA;
- **Porcentagem de jogadas seguras:** Proporção de movimentos que não colocam peças próprias em risco desnecessário.

O aumento na média de jogadas por partida, como visto na Figura 13, sugere que a IA está indo além da simples prevenção de derrotas imediatas. Ela aprende a construir seqüências de jogadas mais complexas, prolongando a partida e indicando um entendimento de estratégia posicional, o que torna o jogo mais desafiador para o oponente humano.

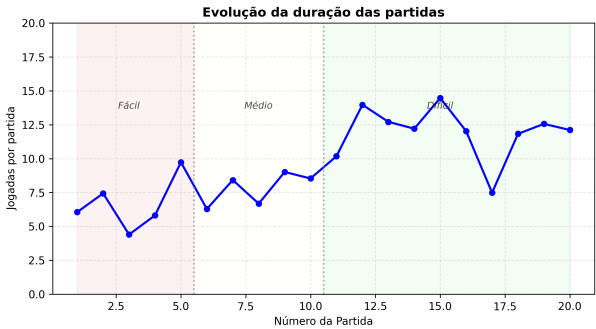


Figura 13. Evolução da média de jogadas por partida ao longo do treinamento.
Fonte: Autoria própria

A análise da porcentagem de jogadas seguras, apresentada na Figura 14, revela melhoria significativa na capacidade da IA de evitar movimentos arriscados. Observa-se crescimento de aproximadamente 60% na taxa de jogadas seguras entre a primeira e a vigésima partida, indicando aprendizado efetivo de princípios básicos de segurança posicional.

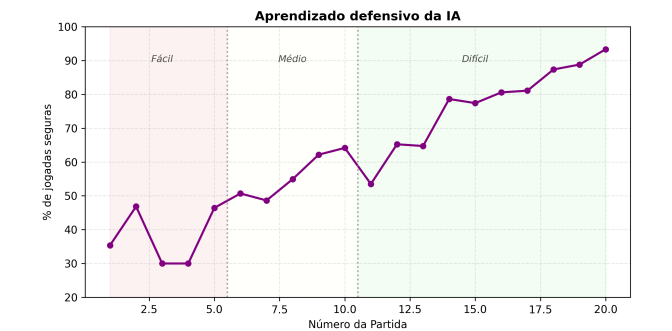


Figura 14. Evolução da porcentagem de jogadas seguras durante o treinamento.
 Fonte: Autoria própria

O saldo de material, mostrado na Figura 15, demonstra a evolução da capacidade da IA de preservar suas peças e capturar peças adversárias de forma estratégica. A tendência crescente evidencia o aprendizado de conceitos fundamentais de valor das peças e trocas vantajosas.



Figura 15. Evolução do saldo de material ao longo das partidas.
 Fonte: Autoria própria

A redução no número de *blunders*, apresentada na Figura 16, confirma a diminuição de erros críticos conforme o treinamento progride. A diferença no número de blunders entre as primeiras e últimas partidas de cada sessão demonstra a eficácia do algoritmo Q-Learning em aprender a evitar movimentos manifestamente prejudiciais.



Figura 16. Redução no número de *blunders* durante o treinamento.
 Fonte: Autoria própria

Os gráficos de desempenho ilustram uma narrativa de aprendizado coesa e progressiva. A evolução da IA começa com uma acentuada redução de *blunders*, Figura 16, estabelecendo uma base defensiva crucial. Essa estratégia é então refinada com o aumento de jogadas seguras, Figura 13, indicando uma transição de uma simples prevenção de erros para um planejamento proativo. A consequência direta dessa evolução é a melhoria no saldo de material, Figura 15, o que confirma que o algoritmo Q-Learning traduz eficazmente a experiência de jogo em uma vantagem tangível e mensurável no tabuleiro.

5.2 Avaliação do Sistema Integrado

A precisão do sistema CNC foi avaliada através de medições da acurácia de posicionamento, apresentando desvio médio inferior a 2mm em relação às posições alvo. Esta precisão mostrou-se adequada para a manipulação das peças de xadrez, não interferindo na jogabilidade.

O módulo de visão computacional demonstrou taxa de acerto de elevada na identificação correta das peças, com os erros concentrados em situações de iluminação extremamente baixa ou alta reflexividade das peças. O tempo médio de processamento de uma imagem foi de 3,2 segundos, concentrando apenas um alto tempo de processamento em tabuleiros cheios.

6. LIMITAÇÕES E TRABALHOS FUTUROS

Durante o desenvolvimento e testes do sistema, foram identificadas algumas limitações que representam oportunidades para melhorias futuras:

Limitações do Hardware: A utilização de dois microcontroladores Arduino UNO aumentou o acoplamento com o computador. A base de MDF apresentou pequenos desníveis que ocasionalmente afetaram a precisão de posicionamento. Diferenças na velocidade de resposta entre os motores de passo geraram pequenas assimetrias na movimentação.

Limitações do Software: O tempo de processamento da visão computacional aumenta significativamente em tabuleiros com muitas peças, limitando a fluidez da interação. O algoritmo Q-Learning, embora eficaz para a variante 4×4, apresenta limitações de escalabilidade para versões mais complexas do jogo.

Trabalhos Futuros: Propõe-se a unificação do controle em um único microcontrolador para realizar todas as tarefas do projeto, simplificando a arquitetura do sistema. A implementação de algoritmos de visão computacional mais eficientes, possivelmente baseados em aprendizado profundo, poderia melhorar a velocidade e precisão do reconhecimento. O desenvolvimento de versões expandidas do jogo, incluindo tabuleiros 6×6 ou 8×8, permitiria observar mais claramente a evolução da IA, uma vez que a variante do xadrez 4x4 é fechada. A integração de elementos de gamificação e sistema de pontuação poderia aumentar o engajamento educacional.

7. CONCLUSÕES

Este trabalho apresentou o desenvolvimento bem-sucedido de um sistema integrado para ensino de conceitos de inteligência artificial através de um jogo de xadrez físico automatizado. O projeto Autochess demonstrou a viabilidade de combinar técnicas de aprendizado de máquina, visão computacional e automação em uma plataforma educacional acessível e interativa.

Os resultados obtidos confirmaram a eficácia do algoritmo Q-Learning na aprendizagem de estratégias básicas de xadrez, com melhoria consistente de desempenho ao longo de múltiplas sessões de treinamento. A implementação do algoritmo SIFT para reconhecimento de peças se mostrou robusta e confiável, apesar do alto tempo de processamento prejudicar a fluidez do jogo.

O custo total de R\$ 418,00 torna o sistema economicamente viável para aplicações educacionais. Apesar disso, a escolha da base de MDF, embora de baixo custo, resultou em um leve envergamento ao longo do tempo, criando desníveis na superfície que exigiram ajustes manuais com pesos para garantir a precisão do braço mecânico.

A arquitetura CoreXY adotada para o sistema CNC proporcionou precisão adequada para a manipulação das peças, enquanto a estrutura modular do projeto facilitou a manutenção e possíveis expansões futuras.

Contudo, é fundamental reconhecer que a arquitetura atual do projeto apresenta uma limitação significativa: o sistema não é *plug and play*. A dependência de um computador executando o código Python para os módulos de Inteligência Artificial e Visão Computacional impede que o dispositivo funcione de maneira autônoma. Essa característica representa uma barreira prática para sua aplicação direta em ambientes educacionais sem a devida preparação técnica, e aponta para a clara necessidade de, em trabalhos futuros, embarcar essa inteligência em um hardware integrado, como um Raspberry Pi, para criar uma solução verdadeiramente independente.

O trabalho contribui para a área de robótica educacional ao integrar múltiplas disciplinas em um único sistema coeso, oferecendo uma ferramenta prática para demonstração de conceitos abstratos de IA. A metodologia desenvolvida pode ser aplicada a outros jogos e contextos educacionais, expandindo as possibilidades de ensino através de sistemas físicos interativos.

Futuras versões do sistema poderão incorporar as melhorias sugeridas, expandindo suas capacidades educacionais

e técnicas. A base sólida estabelecida neste projeto oferece fundamentos para desenvolvimentos mais avançados na intersecção entre educação, inteligência artificial e automação.

AGRADECIMENTOS

Os autores agradecem à disciplina de Oficina de Integração 1 pela oportunidade de desenvolvimento deste projeto, aos professores Juliano Mourão Vieira, Eduardo Nunes dos Santos e Ronnier Frates Rohrich (DAELN) pela orientação, e ao Professor Bogdan Tomoyuki Nassu (DAINF) pelo apoio no desenvolvimento da Visão Computacional. Agradecimentos especiais à Maker Mate pela disponibilização da estrutura física e equipamentos necessários para a construção do sistema.

REFERÊNCIAS

- [1] Tedre, M., Toivonen, T., Kahila, J., Vartiainen, H., Valtonen, T., Jormanainen, I., & Pears, A. (2021). Teaching machine learning in K-12 computing education: Potential and pitfalls. *ACM Transactions on Computing Education (TOCE)*, 21(3), 1-26.
- [2] Ensmenger, N. (2012). Is chess the drosophila of artificial intelligence? A social history of an algorithm. *Social Studies of Science*, 42(1), 5-30.
- [3] Novak, M., & Schwan, S. (2021). Does touching real objects affect learning?. *Educational Psychology Review*, 33(2), 637-665.
- [4] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., ... & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484-489.
- [5] Gardner, M. (1981). *Minichess variants*. *Mathematical Games*, Scientific American, 244(1), 18-22.
- [6] Benitti, F. B. V. (2012). Exploring the educational potential of robotics in schools: A systematic review. *Computers & Education*, 58(3), 978-988.
- [7] Tested 3D Prints. *Homework Writing Machine – Arduino based CoreXY Plotter*. Disponível em: <https://test3dprints.com/arduino/homework-writing-machine/>. Acesso em: 22 jun. 2025.
- [8] MakerC. (2016). *DrawingBot - CoreXY Drawing Robot*. Disponível em: <https://www.thingiverse.com/thing:1517211>. Acesso em: 10 dez. 2024.
- [9] Wikipedia. *Minichess*. Disponível em: <https://en.wikipedia.org/wiki/Minichess>. Acesso em: 22 jun. 2025.
- [10] Robottini. (2020). *GRBL-Servo: CNC Controller with Servo Support*. Disponível em: <https://github.com/robottini/grbl-servo>. Acesso em: 15 dez. 2024.
- [11] Watkins, C. J., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279-292.
- [12] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.
- [13] Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2), 91-110.